



universidad de buenos aires - exactas

departamento de física

laboratorios de enseñanza

Controlador de temperatura para los laboratorios de enseñanza de física

Compendio de I+D, revisiones técnicas y
manual de uso

Marcos Damián Perez

Facultad de Ciencias Exactas y Naturales

Departamento de Física

Licenciatura en Ciencias Físicas

Laboratorios de enseñanza

Buenos Aires, junio 2026

Controlador de temperatura para los laboratorios de enseñanza de física

Compendio de I+D, revisiones técnicas y manual de uso

Marcos Damián Perez

Supervisor: Cobelli, Pablo

Profesor , Departamento de física

Coordinador: Alejandro César Greco

Coordinador, Departamento de física

Pañoleros: Alejandro

Damián

Federico

Juanita

Mateo

Magali

Agustina

Yamila

mas...

Facultad de Ciencias Exactas y Naturales

Departamento de Física

Licenciatura en Ciencias Físicas

Laboratorios de enseñanza

Cuaderno

Buenos Aires, junio 2026

Controlador de temperatura para los laboratorios de enseñanza de física

Copyright © 2026 - Marcos Damián Perez, Facultad de Ciencias Exactas y Naturales.

Este trabajo ... descargo correspondiente.

Agradecimientos

Guía de Escritura

Agradecimientos a quien corresponda.

1

Resumen

Documentacion del controlador de temperatura desarrollado para laboratorio 4 de la carrera de licenciatura en fisica de la universidad de buenos aires. El documento incluye una descripción detallada del diseño, desarrollo y pruebas del controlador, así como un análisis de su desempeño y posibles mejoras futuras. El objetivo principal es proporcionar una herramienta para la medición y control de la temperatura en experimentos de laboratorio, con un enfoque en la precisión, estabilidad y facilidad de uso. El controlador de temperatura desarrollado es capaz de mantener una temperatura constante con una alta precisión, lo que lo hace ideal para una amplia gama de aplicaciones en el laboratorio.

Índice general

1. Resumen	III
<i>Índice de figuras</i>	VIII
<i>Índice de cuadros</i>	X
<i>Glosario</i>	XII
<i>Siglas</i>	XIV
<i>Símbolos</i>	XVI
I Estado Actual del Sistema	1
2. Especificaciones	2
2.1. Actualizacion, 2 de junio de 2026	2
2.2. historial de cambios:	2
3. Arquitectura General del Amplificador	3
3.1. El controlador de temperatura	3
3.1.1. Estado del arte	3
3.1.2. Etapa de salida	4
3.1.3. Medición de corriente y tensión	4
3.1.4. Sistemas de protección	4
3.2. El panel de frontal	4
3.2.1. Controles	4
3.2.2. Indicadores	4
3.2.3. Display	4
3.3. Interfaz de control remoto y programación	4
3.3.1. Interfaces de comunicación	4
3.3.2. Protocolos de comunicación	4
3.3.3. Comandos SCPI	4
4. Controlador-de-Temperatura	5
5. Control y Medición	6

6. Protecciones	7
7. Alimentación	8
8. Interfaz de Usuario	9
9. Diseño Térmico	10
10. Diseño Mecánico	11
11. Integración Electromecánica	12
12. Implementacion Electronica	13
12.1. Esquematicos	13
12.2. PCBs	13
13. Implementacion Mecanica	14
14. Implementación de Firmware	15
14.1. dependencias	15
15. Validación Eléctrica	16
16. Validación Térmica	17
17. Validación del Sistema	18
18. Manual de Operación	19
19. Mantenimiento	20
 II Gestión de Versiones del Producto	 21
20. Historial de Versiones	22
 III Desarrollo de Firmware	 23
21. Desarrollo de firmware	24
21.1. Resumen	24
22. Sistema de Comunicación	25
22.1. Arquitectura General	25
22.1.1. Modelo de Capas y Responsabilidades	25
22.1.2. Propiedad de Recursos	27
22.1.3. Conceptos Fundamentales	30
22.1.4. Flujo de Comunicación	32

22.2. Capa de Driver	34
22.2.1. Interfaz Abstracta <code>comm_iface_t</code>	34
22.2.2. Estados de la Interfaz	36
22.2.3. Eventos de Comunicación y Error	37
22.2.4. APIs de Control de Flujo	39
22.2.5. Protección de Acceso Concurrente	40
22.2.6. Estrategias de Sincronización y Gestión de Buffers	42
22.3. Capa de Middleware (Communication Task)	43
22.3.1. Modelo de Ejecución y Planificación	43
22.3.2. Registro de Eventos y Captura Atómica (Snapshot)	44
22.3.3. Orden de Precedencia y Flujo de Control	44
22.3.4. Máquinas de Estado de Transporte	45
22.3.5. API de Servicios hacia la Capa de Aplicación	46
22.3.6. Adaptación a Protocolos de Instrumentación	47
22.3.7. Selector de Medio de Transporte	47
23. Motor SCPI	49
23.1. Integración del parser <code>libscpi</code>	49
23.2. Flujo de recepción y análisis de comandos	49
23.3. Generación y transmisión de respuestas	49

Anexos

Índice de figuras

22.1. Modelo de capas del sistema de comunicación. Las flechas indican las dependencias funcionales y el sentido del flujo de datos.	25
22.2. Estructura <code>comm_iface_t</code> y su interacción con la <i>Communication Task</i> y el driver concreto.	35

Índice de cuadros

22.1. Propiedad de recursos en el sistema de comunicación.	27
22.2. Estados definidos en <code>comm_iface_state_t</code>	36
22.3. Valores definidos en <code>com_response_t</code>	39
22.4. Tabla de transiciones de la FSM de Gestión de Interfaz.	45
22.5. Tabla de transiciones de la FSM de Recepción.	46
22.6. Tabla de transiciones de la FSM de Transmisión.	46

Parte I

Estado Actual del Sistema

2

Especificaciones

2.1 Actualizacion, 2 de junio de 2026

Especificaciones del sistema en desarrollo. Estas especificaciones pueden ser modificadas a lo largo del proyecto, pero es importante tener una base clara desde el principio para guiar el diseño y desarrollo del controlador de temperatura.

- Seguridad: El diseño debe cumplir con las normativas de seguridad del laboratorio, evitando riesgos para los estudiantes y el equipo.
- Corriente máxima de salida: 10 A continuos (DC).
- Tensión máxima de salida: 24 V.
- Ancho de banda: DC a 10 Hz.
- Precisión en DC: offset menor a 5 mV (si puede ser menos, mejor).
- Ruido de salida menor a 10 mV
- Sistemas de protección: El diseño debe incluir protecciones contra sobrecorriente, sobretensión, cortocircuitos y sobrecalentamiento para garantizar la seguridad y la durabilidad del equipo.
- Interfaz de control: El controlador debe ser controlable a través de una interfaz digital (por ejemplo, USB, Ethernet o RS-232) para permitir su integración con sistemas de adquisición de datos y controladores externos.
- El equipo debe soportar comandos SCPI para facilitar su integración con software de control y adquisición de datos comúnmente utilizado en laboratorios.
- Interfaz de usuario: El Controlador debe contar con una interfaz de usuario intuitiva para facilitar su operación por parte de los estudiantes y el personal del laboratorio, incluyendo indicadores de estado y controles de ajuste de la salida.

2.2 historial de cambios:

- **2 de junio del 2026:** Se establecieron las especificaciones del sistema en desarrollo.

3

Arquitectura General del Amplificador

En esta sección se presenta una descripción general de la arquitectura del amplificador lineal de potencia, destacando los componentes principales y su función dentro del sistema. Se abordarán aspectos como la etapa de entrada, la etapa de amplificación, la etapa de salida y los sistemas de protección y control.

3.1 El controlador de temperatura

Actualmente para no hay en el alboratorio un controlador de temperatura y solo se deja evolucionar la temperatura midiendo en los trancitorios y estacionarios. El objetivo de este proyecto es desarrollar un controlador de temperatura que permita mantener una temperatura constante con alta precisión, lo que es fundamental para una amplia gama de experimentos en el laboratorio, no solo para tener la bariable controlada sino tambien para acelerar los largo transitorios de termalización de los experimentos. El controlador de temperatura desarrollado se integrará con el sistema de adquisición de datos del laboratorio, permitiendo un control preciso y una fácil monitorización de la temperatura durante los experimentos.

3.1.1 Estado del arte

- Etapa de salida: Puentes H con transistores MOSFET de potencia en configuración push-pull para manejar las altas corrientes y tensiones requeridas, con un diseño que minimice las pérdidas de conmutación y maximice la eficiencia.
- Control de la etapa de salida: modulación por ancho de pulso (PWM) para controlar la potencia entregada a la carga, con un controlador de retroalimentación para mantener la temperatura constante.
- Filtro de salida: Un filtro LC para suavizar la señal de salida y reducir el ruido, asegurando una entrega de potencia estable y precisa a la carga.

- Control y medición: Un microcontrolador o FPGA para implementar el control del amplificador, la adquisición de datos de los sensores de corriente y tensión, y la comunicación con la interfaz de usuario.
- Sistemas de protección: Circuitos de protección contra sobrecorriente, sobreten-
sión, cortocircuitos y sobrecalentamiento para garantizar la seguridad y la dura-
bilidad del equipo.
- Medición de corriente y tensión: sensores de corriente y tensión de alta precisión para monitorear la salida del amplificador y proporcionar retroalimentación al controlador.
- Interfaz de usuario: Pantalla LCD o LED para mostrar el estado del amplificador, controles de ajuste para configurar la salida y botones para encender/apagar el dispositivo.

3.1.2 Etapa de salida

3.1.3 Medición de corriente y tensión

3.1.4 Sistemas de protección

3.2 El panel de frontal

3.2.1 Controles

3.2.2 Indicadores

3.2.3 Display

3.3 Interfaz de control remoto y programación

3.3.1 Interfaces de comunicación

3.3.2 Protocolos de comunicación

3.3.3 Comandos SCPI

4

Controlador-de-Temperatura

5

Control y Medición

6

Protecciones

7

Alimentación

8

Interfaz de Usuario

9

Diseño Térmico

10

Diseño Mecánico

11

Integración Electromecánica

12

Implementación electrónica

12.1 Esquematicos

12.2 PCBs

13

Implementación mecánica

14

Implementacion de Firmware

14.1 dependencias

-

15

Validación Eléctrica

16

Validación Térmica

17

Validación del Sistema

18

Manual de Operación

19

Mantenimiento

Parte II

Gestión de Versiones del Producto

20

Historial de Versiones

Parte III

Desarrollo de Firmware

21

Desarrollo de firmware

21.1 Resumen

Estructura y organización y resumen del desarrollo:

Sistema de comunicación: Driver Dependencias y obligaciones API Inyección de dependencias (buffers, etc) Communication Task Mantenimiento de interfaz Manejo de errores Recepción y extracción de mensajes Transmisión API Motor SCPI Búsqueda de comandos Parser SCPI Control de errores Armado de Respuestas Escritura de mensajes de respuesta

Panel Frontal: Pantalla API Control API

Control: Salida de potencia Seguridad API Control PID Modelo incremental para DSP Sensores de temperatura Filtros API

22

Sistema de Comunicación

22.1 Arquitectura General

22.1.1 Modelo de Capas y Responsabilidades

El sistema de comunicación se organiza en tres capas funcionales con responsabilidades estrictamente delimitadas y fronteras bien definidas mediante interfaces abstractas. Esta segmentación permite modificar o reemplazar cualquiera de las capas sin impactar a las demás, siempre que se respeten los contratos establecidos.

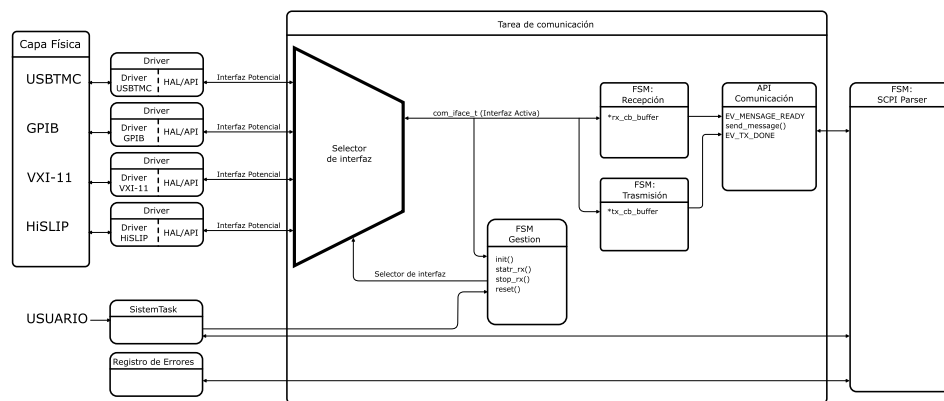


Figura 22.1: Modelo de capas del sistema de comunicación. Las flechas indican las dependencias funcionales y el sentido del flujo de datos.

Capa de Driver (Interfaz de Transporte Físico) Es la capa más cercana al hardware. Su responsabilidad exclusiva es la gestión del periférico de comunicación (UART, USBTMC, SPI, TCP/IP, etc.) y la traducción de eventos físicos a una representación abstracta normalizada.

El driver expone sus servicios a través de la interfaz `comm_iface_t`, que constituye un contrato cerrado. Toda implementación concreta debe cumplir con esta interfaz para ser utilizable por las capas superiores. Esta abstracción oculta por completo las

particularidades del registro de periférico, las interrupciones, el DMA o los protocolos de enlace de bajo nivel.

La capa de driver:

- Administra los recursos de hardware (registros, interrupciones, canales DMA).
- Produce eventos abstractos de interfaz (dato recibido, transmisión completada, error).
- Mantiene el estado operativo de la conexión física.
- No asigna buffers de aplicación ni realiza procesamiento de los datos transportados.

Capa de Middleware (Communication Task) Esta capa constituye el núcleo del sub-sistema de comunicación. Se implementa como una tarea independiente (o un bucle dentro de un esquema cooperativo) y opera exclusivamente sobre la interfaz abstracta proporcionada por el driver.

Sus responsabilidades comprenden:

- La gestión global del ciclo de vida de la interfaz (inicialización, arranque, detención, recuperación ante fallos).
- La recepción de bytes desde el driver y la delimitación de mensajes completos de acuerdo con las reglas del protocolo de instrumentación (detección del terminador LF según IEEE 488.2).
- La transmisión de mensajes hacia el driver cuando la aplicación lo solicita.
- La publicación de eventos lógicos hacia la capa de aplicación: EV_MESSAGE_READY (mensaje disponible) y EV_TX_DONE (transmisión finalizada).
- La coordinación de las máquinas de estado de gestión, recepción y transmisión.

El middleware desconoce por completo el contenido semántico de los mensajes que transporta. Un mensaje es, para esta capa, una secuencia de bytes delimitada por un terminador; no realiza análisis sintáctico ni interpreta comandos.

Capa de Aplicación (Motor SCPI) Es la capa superior, consumidora de los servicios de comunicación. Recibe mensajes completos desde el middleware y los procesa de acuerdo con la sintaxis y semántica del protocolo SCPI (Standard Commands for Programmable Instruments).

Sus responsabilidades son:

- El análisis sintáctico y semántico de los comandos recibidos.
- La ejecución de las acciones solicitadas (consulta de medidas, modificación de parámetros, etc.).
- La generación de respuestas conformes al estándar IEEE 488.2.
- La interacción con el resto del sistema del instrumento (control, medición, configuración).

El motor SCPI no accede directamente al hardware de comunicación ni a los buffers de transporte. Para enviar una respuesta, invoca la función `send_message()` expuesta por el middleware.

Principio de Propiedad de Recursos Cada capa es propietaria exclusiva de los recursos que administra. Este principio evita dependencias implícitas y condiciones de carrera:

- Los buffers de transporte (recepción y transmisión) son propiedad de la *Communication Task* y se inyectan al driver por referencia durante la inicialización. El driver opera sobre ellos pero no los posee.
- El contexto del periférico (registros de configuración, estado interno del driver) es privado de cada implementación concreta del driver y no es accesible desde fuera.
- Los buffers de comandos y respuestas SCPI pertenecen al motor SCPI y son independientes del transporte.

La tabla 22.1 detalla la asignación de propiedad para cada recurso compartido.

Cuadro 22.1: *Propiedad de recursos en el sistema de comunicación.*

Recurso	Propietario	Observación
Buffer RX de transporte	Communication Task	Asignado a la estructura <code>comm_iface_t</code>
Buffer TX de transporte	Communication Task	Asignado a la estructura <code>comm_iface_t</code>
Contexto del periférico	Driver	Privado de cada implementación
Registro de eventos	Driver	Compartido ISR/Task mediante snapshot
Estado de interfaz	Driver	Visible para capas superiores en modo lectura
Buffers de comandos SCPI	Motor SCPI	Independientes del transporte
Buffers de respuesta SCPI	Motor SCPI	Independientes del transporte

Flujo de Información entre Capas La comunicación entre capas sigue un modelo orientado a eventos, sin llamadas bloqueantes. El sentido del flujo es el siguiente:

Recepción: Medio físico → Driver → Eventos de interfaz → Communication Task → Mensaje completo → Motor SCPI.

Transmisión: Motor SCPI → Solicitud de transmisión → Communication Task → Driver → Medio físico.

La capa de aplicación (Motor SCPI) se documenta en detalle en el capítulo siguiente. El resto de este capítulo describe la arquitectura interna de las capas de Driver y Middleware, así como las interfaces que las conectan.

22.1.2 Propiedad de Recursos

La asignación explícita de la propiedad de cada recurso compartido es un principio arquitectónico fundamental del sistema de comunicación. Establecer un propietario

único para cada estructura de datos, buffer o registro de estado persigue tres objetivos:

1. **Eliminar la contención implícita:** cuando dos módulos pueden modificar un mismo recurso sin una mediación clara, aparecen condiciones de carrera y efectos laterales difíciles de depurar.
2. **Permitir la verificación estática:** el propietario es responsable de asignar, liberar y garantizar la validez del recurso; los demás módulos lo utilizan bajo un contrato de acceso definido (lectura, escritura mediada, etc.).
3. **Facilitar la portabilidad:** si cada recurso tiene un dueño bien identificado, reemplazar una capa (por ejemplo, cambiar el driver de USBTMC por uno de TCP/IP) no obliga a modificar la gestión de memoria de las demás capas.

La tabla 22.1 (presentada en la sección anterior) resume la asignación de propiedad para los recursos principales. A continuación se detallan las implicancias de cada caso.

Buffers de Transporte (RX y TX) Los buffers de recepción y transmisión a nivel de transporte son **propiedad exclusiva de la *Communication Task***. Esto significa que:

- La tarea de comunicación reserva estáticamente la memoria necesaria (por ejemplo, en su propio stack o en la sección `.bss` de su módulo objeto).
- Durante la inicialización, las referencias a estos buffers se asignan directamente en los campos `rx_buffer` y `tx_buffer` de la estructura `comm_iface_t` antes de invocar `init()`.
- El driver puede leer y escribir en ellos, pero **no es responsable de su ciclo de vida**. Si un driver necesita dimensionar los buffers, lo hace a través de parámetros de configuración, no asignándolos por sí mismo.
- Esta separación permite que, en un mismo hardware, dos interfaces distintas compartan la misma tarea de comunicación sin que el driver imponga un tamaño fijo de buffer.

Contexto del Periférico Cada implementación concreta del driver mantiene una estructura de contexto privada (registros de configuración, estados internos, handles de sistema operativo, etc.). Este contexto es **propiedad exclusiva del driver** y no es accesible desde la *Communication Task* ni desde la aplicación. Las capas superiores desconocen por completo su contenido y solo interactúan con el driver a través de la interfaz abstracta `comm_iface_t`.

Esta encapsulación garantiza que:

- El middleware no pueda, por error, modificar un registro de periférico sin pasar por la API.
- Se pueda implementar un driver simulado (*mock*) para pruebas unitarias del middleware, reemplazando completamente el contexto real sin que la tarea de comunicación lo advierta.

Registro de Eventos El registro de eventos es una estructura de mapa de bits (*bit-mask*) que el driver actualiza de forma asíncrona (generalmente desde una ISR) y la *Communication Task* lee en cada ciclo. La **propiedad formal** reside en el driver, pero el acceso es compartido:

- Al inicio del siguiente ciclo de la *Communication Task*, se captura atómicamente el registro de eventos del driver en una variable local (*snapshot*) y se limpia el registro original. A partir de este momento, todas las FSM operan sobre la copia local, cada una enmascarando los bits que le corresponden.
- La *Communication Task* lee el registro de manera atómica (mecanismo de *snapshot* descrito en la sección 22.3.2) y luego lo limpia.
- Ninguna otra capa accede directamente a este registro, ni siquiera en modo lectura. La aplicación recibe eventos lógicos de más alto nivel a través de la API de comunicación (EV_MESSAGE_READY, EV_TX_DONE).

Estado de la Interfaz El estado operativo de la interfaz (COMM_STATE_CONNECTED, COMM_STATE_RX_ACTIVE etc.) es mantenido por el driver y **visible en modo lectura** para la *Communication Task*. La tarea de middleware puede consultar este estado para tomar decisiones de control (por ejemplo, no intentar transmitir si la interfaz no está conectada), pero no puede modificarlo directamente. La modificación del estado es consecuencia de las operaciones invocadas a través de la API (start_rx(), stop_rx()) o de eventos de hardware detectados por el driver.

Buffers de Comandos y Respuestas SCPI Los buffers donde se almacenan los comandos recibidos (ya delimitados) y las respuestas generadas por el instrumento son **propiedad exclusiva del Motor SCPI**. Esta decisión es crítica para la independencia de la aplicación respecto al transporte:

- El middleware entrega el mensaje completo a la capa de aplicación copiándolo en el buffer de comandos SCPI o pasando una referencia que la aplicación debe consumir antes del siguiente ciclo de la tarea.
- La aplicación formatea sus respuestas en sus propios buffers y luego invoca send_message() para transferirlas al middleware.
- Si en el futuro se reemplaza el protocolo SCPI por otro (por ejemplo, Modbus), los buffers de la aplicación cambian, pero el middleware de comunicación permanece inalterado.

Principio de Inyección de Dependencias La separación de propiedad se complementa con un patrón de **inyección de dependencias** en tiempo de inicialización. La *Communication Task* no instancia directamente un driver concreto, sino que recibe una referencia a una implementación de comm_iface_t junto con los parámetros de configuración necesarios (incluyendo las referencias a los buffers de transporte). Este meca-

nismo, detallado en la sección 22.2.6, es el que permite que el mismo código de middleware opere sobre UART, USBTMC o TCP/IP sin modificaciones.

22.1.3 Conceptos Fundamentales

Para garantizar una interpretación unívoca de la arquitectura, esta sección define los términos que se emplean de manera recurrente en la documentación del sistema de comunicación. Cada concepto está acotado al contexto de este subsistema; otros módulos del firmware pueden utilizar definiciones complementarias.

Estado Un estado representa una condición operativa persistente de una interfaz de comunicación. Describe una situación que se mantiene durante un intervalo de tiempo y que puede consultarse en cualquier momento.

En el sistema de comunicación, los estados se implementan como una máscara de bits (`comm_iface_state_t`), lo que permite que múltiples condiciones coexistan simultáneamente. Por ejemplo, una interfaz puede estar al mismo tiempo `CONNECTED` y `RX_ACTIVE`.

Los estados son modificados exclusivamente por el driver como resultado de operaciones solicitadas a través de la API (`start_rx()`, `stop_rx()`) o de eventos de hardware detectados internamente. Las capas superiores pueden leerlos, pero no alterarlos directamente.

Evento Un evento es la representación de un suceso instantáneo y asincrónico, producido por el hardware de comunicación o por el propio driver. A diferencia de los estados, un evento no describe una condición permanente sino una notificación puntual que debe ser procesada por las capas superiores.

Los eventos se codifican como una máscara de bits (`comm_iface_event_t`) en la que cada bit representa un tipo de suceso. Esta representación permite:

- La coexistencia de múltiples eventos simultáneos.
- El filtrado eficiente mediante operaciones de enmascaramiento binario.
- La transferencia atómica del registro completo (mecanismo de *snapshot*) entre el contexto de interrupción y el de tarea.

Ejemplos típicos de eventos son la disponibilidad de nuevos datos en el buffer de recepción, la finalización de una transmisión o la detección de una condición de error de hardware.

Error Un error es una condición anómala detectada durante la operación de la interfaz de comunicación. Los errores pueden originarse en el medio físico (errores de paridad, de encuadre, desbordamiento del buffer de hardware), en la capa de transporte (timeout de transmisión, fallo de bus) o en la propia implementación del driver (error interno).

La ocurrencia de un error se notifica mediante los eventos específicos definidos en `comm_iface_event_t` (`IFACE_EVENT_RX_ERROR_OVERFLOW`, `IFACE_EVENT_TX_ERROR_TIMEOUT`, etc.). Adicionalmente, un error crítico puede provocar la activación del estado persistente `COMM_STATE_ERROR`, lo que indica a las capas superiores que la interfaz no se encuentra en condición operativa normal.

Es importante distinguir entre el código de error retornado por algunas funciones de la API (como `init()` o `reset()`) y los eventos de error asincrónicos. Los primeros informan del resultado de una operación solicitada; los segundos notifican condiciones sobrevenidas fuera del flujo de llamadas.

Operación Una operación es una acción explícita solicitada por una capa superior a través de la API de la interfaz de comunicación. Constituye el mecanismo de control de que disponen la *Communication Task* y, de forma mediada, la aplicación, para gobernar el comportamiento del driver sin exponer los detalles de implementación del periférico.

Las operaciones se corresponden con los *callbacks* definidos en la estructura `comm_iface_t`:

- **Ciclo de vida:** `init()`, `deinit()`, `reset()`.
- **Control de flujo:** `start_rx()`, `stop_rx()`, `start_tx()`.
- **Gestión de eventos:** `set_event()`, `get_event()`.
- **Protección concurrente:** `protect_rx()`, `unprotect_rx()`, `protect_tx()`, `unprotect_tx()`.

Todas las operaciones están diseñadas para ser no bloqueantes, en coherencia con el modelo de ejecución cooperativa de la *Communication Task*.

Contexto El contexto es una estructura de datos privada asociada a una instancia concreta de interfaz de comunicación. En la implementación, la estructura `comm_iface_t` incluye un campo `void *context` que cada driver utiliza para almacenar la información que necesita: registros de configuración del periférico, handles del sistema operativo, punteros a buffers internos, etc.

Ni la *Communication Task* ni la capa de aplicación acceden al contenido del contexto; solo lo pasan como argumento en cada invocación a los *callbacks* del driver. Esta opacidad garantiza la independencia de las capas superiores respecto de los detalles de implementación del hardware.

Propietario El propietario de un recurso es el componente del sistema responsable de:

- Reservar la memoria o el handle que lo representa.
- Garantizar su validez durante el tiempo de uso.
- Liberarlo cuando ya no es necesario.

La asignación de propiedad está explícitamente documentada en la tabla 22.1. Los componentes que no son propietarios de un recurso pueden utilizarlo únicamente bajo

las condiciones establecidas por el contrato de acceso (por ejemplo, el driver escribe en los buffers de transporte inyectados por la *Communication Task*, pero no es responsable de su asignación ni de su liberación).

Mensaje En el contexto del sistema de comunicación, un mensaje es la unidad de información que se transfiere entre la *Communication Task* y la capa de aplicación. Se define como una secuencia contigua de bytes, delimitada por un carácter de fin de mensaje (el terminador LF, 0x0A, conforme a IEEE 488.2), y despojada de cualquier carácter de control de transporte (como CR, 0x0D).

La delimitación del mensaje es responsabilidad de la máquina de estados de recepción del middleware. Una vez ensamblado el mensaje completo, se notifica a la aplicación mediante el evento lógico EV_MESSAGE_READY. La aplicación puede entonces consumir el mensaje desde el buffer de recepción de la *Communication Task* (o copiarlo a sus buffers propios) y proceder a su análisis sintáctico.

Análogamente, cuando la aplicación desea transmitir información, compone un mensaje en sus buffers de respuesta y solicita el envío invocando `send_message()`, lo que activa la máquina de estados de transmisión del middleware.

Esta definición desacopla completamente el transporte del contenido: el middleware no distingue entre un comando SCPI, una consulta de estado o cualquier otra carga útil; opera exclusivamente con mensajes como secuencias de bytes delimitadas.

22.1.4 Flujo de Comunicación

El modelo de interacción entre capas es puramente asíncrono y orientado a eventos. No existen llamadas bloqueantes ni dependencias cíclicas entre la *Communication Task* y la capa de aplicación. La secuencia de operaciones se describe a continuación para los dos escenarios fundamentales: recepción de un mensaje y transmisión de una respuesta.

Flujo de Recepción

1. El medio físico entrega datos al periférico. La ISR del driver o el DMA transfieren los bytes al buffer de recepción inyectado (`rx_buffer`, propiedad de la *Communication Task*) utilizando la estructura de buffer circular y respetando las protecciones definidas por el driver.
2. El driver activa en su registro de eventos el bit correspondiente a `IFACE_EVENT_RX_DATA_AVAILABLE`. Si se detecta una condición de error durante la recepción (overflow, framing, paridad), se activa el bit de error específico en lugar del de datos, o simultáneamente con él.
3. Al inicio del siguiente ciclo de la *Communication Task*, se captura atómicamente el registro de eventos del driver en una variable local (*snapshot*) y se limpia el registro original. A partir de este momento, todas las FSM operan sobre la copia local, cada una enmascarando los bits que le corresponden.

4. La FSM de Recepción consume los eventos del *snapshot*. Si detecta `IFACE_EVENT_RX_DATA_AVAILABLE` extrae uno a uno los bytes directamente de su buffer `rx_buffer` mediante la función `cb_get()` de la biblioteca de buffer circular.
5. Cada byte es evaluado contra el terminador LF (0x0A). Los caracteres CR (0x0D) se descartan silenciosamente. Mientras no aparezca LF, la máquina permanece en el estado `RX_GATHERING`.
6. Al detectar el terminador, la FSM de Recepción considera el mensaje completo, retorna al estado `RX_IDLE` y publica el evento lógico interno `EV_MESSAGE_READY`.
7. La capa de aplicación (Motor SCPI), que es notificada de este evento en el mismo ciclo o en el siguiente según la implementación, toma el mensaje desde el buffer de recepción del middleware y procede a su análisis sintáctico.

Flujo de Transmisión

1. La capa de aplicación, una vez generada una respuesta en sus buffers propios, invoca la función `send_message()` provista por la API de la *Communication Task*, pasando la referencia al buffer de respuesta y su longitud.
2. La *Communication Task* copia los bytes desde el buffer de la aplicación hacia su buffer de transmisión (`tx_buffer`) utilizando la función `cb_put()` de la biblioteca de buffer circular. Una vez copiados los datos, activa internamente el evento lógico `EV_TX_REQUEST`.
3. La FSM de Transmisión, al evaluar este evento en su ciclo correspondiente, transita al estado `TX_BUSY` e invoca directamente `start_tx()` sobre la interfaz activa.
4. El driver, en su implementación de `start_tx()`, verifica el estado del hardware y la disponibilidad de datos en `tx_buffer`:
 - Si el hardware está ocupado, retorna `COMM_IFACE_BUSY`. La FSM permanece en `TX_BUSY` y reintentará en el siguiente ciclo.
 - Si el buffer está vacío, retorna `COMM_IFACE_IDLE`. La FSM retorna a `TX_IDLE` sin realizar acción alguna.
 - Si el hardware está en condición de error, retorna `COMM_IFACE_ERROR`. La FSM transita a `TX_IDLE` y propaga el error a la aplicación.
 - Si el hardware acepta la transmisión, retorna `COMM_IFACE_OK`. La FSM permanece en `TX_BUSY` a la espera del evento de finalización.
5. Cuando el driver completa la transmisión (por DMA o interrupción), activa en su registro de eventos el bit `IFACE_EVENT_TX_COMPLETE`.
6. En el siguiente ciclo de la tarea de comunicación, la FSM de Transmisión captura este evento desde el *snapshot*, retorna al estado `TX_IDLE` y publica el evento lógico `EV_TX_DONE` hacia la aplicación.
7. Si durante la transmisión se produce un error (timeout, fallo de bus), el driver activa el evento de error correspondiente (`IFACE_EVENT_TX_ERROR...`) y, si corresponde, activa el estado `COMM_STATE_ERROR`. La FSM de Transmisión, al detectar el evento de error en el *snapshot*, puede ejecutar `reset()` sobre el canal de

transmisión y notificar el error a la aplicación a través de la API de comunicación.

Sincronización y No Bloqueo Todo el ciclo de recepción y transmisión se ejecuta dentro del mismo hilo de la *Communication Task*, sin necesidad de semáforos ni colas adicionales. La aplicación, si se ejecuta en el mismo contexto (esquema cooperativo), procesa los mensajes de forma secuencial después de que la tarea de comunicación complete su ciclo. Si la aplicación reside en una tarea separada, la transferencia se realiza mediante una cola de mensajes —implementada como un buffer circular simple— que garantiza la entrega sin bloqueo.

Este flujo será refinado con las máquinas de estado concretas en la sección de Middleware, pero la descripción aquí presentada constituye el marco de referencia para todo el subsistema.

22.2 Capa de Driver

La capa de driver constituye el punto de contacto entre el *hardware* de comunicación y el *firmware* de nivel superior. Su misión es traducir las particularidades de cada periférico físico a una representación abstracta y homogénea, de modo que el resto del sistema pueda operar sin conocer los detalles de implementación del medio de transporte.

Esta sección describe el contrato que todo driver debe satisfacer —la interfaz abstracta `comm_iface_t`—, los recursos que administra y las estrategias de sincronización que emplea para operar de manera segura en un entorno con concurrencia.

22.2.1 Interfaz Abstracta `comm_iface_t`

La interfaz `comm_iface_t` es una estructura de punteros a función y campos de datos que define el contrato vinculante para cualquier implementación de driver de comunicación. Está diseñada para ser completamente agnóstica respecto del periférico subyacente: UART, USBTMC, TCP/IP o cualquier otro medio pueden ser encapsulados detrás de esta misma interfaz.

La estructura, definida en el código como `comm_iface_t`, contiene los siguientes grupos funcionales:

- **Identificación:** un campo `id` (`comm_iface_id_t`) que identifica unívocamente la interfaz (UART, TCP, USBTMC) y un campo `name` legible para diagnóstico.
- **Contexto privado:** un puntero opaco `void *context` que cada implementación concreta utiliza para almacenar sus datos internos (registros del periférico, handles, buffers auxiliares). Ni la *Communication Task* ni la aplicación acceden a este campo; simplemente lo reenvían en cada invocación a los *callbacks*.
- **Estado y eventos:** los campos `state` (`comm_iface_state_t`) y `event` (`comm_iface_event_t`) mantienen, respectivamente, la condición operativa actual de la interfaz y el registro de eventos pendientes de procesar.

- **Buffers de transporte inyectados:** punteros `rx_buffer` y `tx_buffer` a sendos *buffers circulares*. Estos buffers son propiedad de la *Communication Task* y se asignan a la estructura antes de invocar `init()`. El driver los utiliza como espacio de trabajo para la transferencia de datos, pero no es responsable de su asignación ni de su liberación.
- **Callbacks de ciclo de vida:** `init()`, `deinit()` y `reset()`. Gestionan la inicialización, desinicialización y reinicio del periférico. Retornan un código de error de tipo `comm_error_t` para informar del resultado de la operación.
- **Callbacks de control de flujo:** `start_rx()`, `stop_rx()` y `start_tx()`. Activan y desactivan la recepción, e inician la transmisión de un bloque de datos previamente cargado.
- **Callbacks de gestión de eventos:** `set_event()` y `get_event()`. Permiten escribir y leer el registro de eventos de la interfaz. `get_event()` es utilizado por la *Communication Task* para obtener el *snapshot* atómico al inicio de cada ciclo.
- **Callbacks de protección concurrente:** `protect_rx()`, `unprotect_rx()`, `protect_tx()` y `unprotect_tx()`. Proveen mecanismos de exclusión mutua para el acceso a los buffers de recepción y transmisión desde contextos concurrentes (ISR y tarea). La implementación concreta de estos *callbacks* depende del hardware y del sistema operativo subyacente; las capas superiores los invocan sin conocer su estrategia interna.

La figura 22.2 muestra un diagrama esquemático de la estructura `comm_iface_t` y su relación con los componentes que la utilizan.

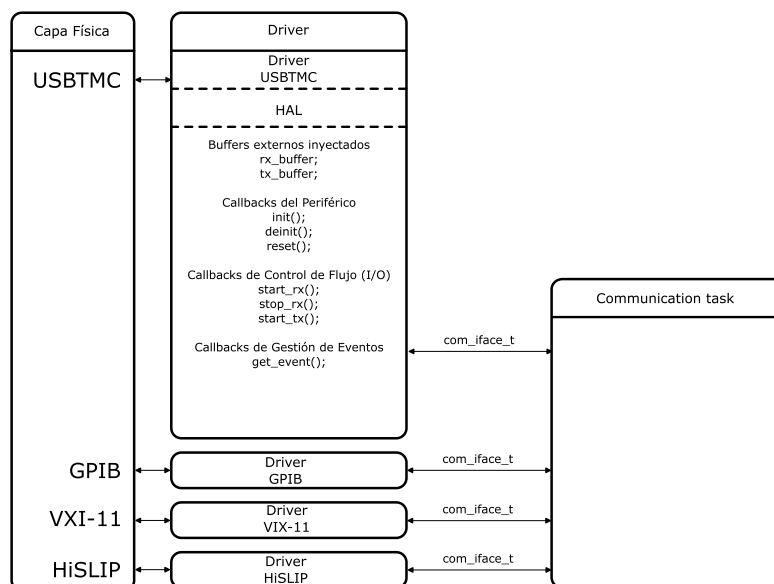


Figura 22.2: Estructura `comm_iface_t` y su interacción con la *Communication Task* y el driver concreto.

La interfaz `comm_iface_t` es el único punto de acoplamiento entre el middleware y el hardware de comunicaciones. Cualquier driver que implemente correctamente estos

callbacks y respete la semántica de los campos de estado y eventos podrá ser utilizado por la *Communication Task* sin modificar una sola línea de su código.

22.2.2 Estados de la Interfaz

La condición operativa de una interfaz de comunicación se representa mediante el campo `state` de la estructura `comm_iface_t`. Este campo es una máscara de bits cuyo tipo, `comm_iface_state_t`, define los siguientes valores:

Cuadro 22.2: Estados definidos en `comm_iface_state_t`.

Símbolo	Máscara	Significado
<code>COMM_STATE_NONE</code>	<code>0x00</code>	Sin estado activo. Interfaz no inicializada o en reposo total.
<code>COMM_STATE_CONNECTED</code>	<code>0x01</code>	Enlace físico o lógico establecido.
<code>COMM_STATE_RX_ACTIVE</code>	<code>0x02</code>	Recepción habilitada. Puede haber datos entrantes o la interfaz está escuchando.
<code>COMM_STATE_TX_ACTIVE</code>	<code>0x04</code>	Transmisión en curso. El periférico o el DMA está enviando datos.
<code>COMM_STATE_ERROR</code>	<code>0x08</code>	Interfaz en condición de falla global persistente.

Estados como condiciones persistentes Un estado permanece activo hasta que una operación explícita o un nuevo evento de hardware provoquen su modificación. Esto diferencia a los estados de los eventos: mientras que un evento notifica un suceso instantáneo (y es consumido por la *Communication Task*), un estado refleja la situación actual de la interfaz y puede ser consultado en cualquier momento.

Coexistencia de estados Al tratarse de una máscara de bits, varios estados pueden estar activos simultáneamente, permitiendo representar condiciones compuestas. Por ejemplo, una interfaz puede encontrarse en los estados `COMM_STATE_CONNECTED` y `COMM_STATE_RX_ACTIVE` al mismo tiempo, indicando que el enlace está establecido y que además se está recibiendo información.

La combinación `COMM_STATE_CONNECTED | COMM_STATE_TX_ACTIVE` describe una interfaz que está transmitiendo datos sobre un enlace activo.

Semántica de cada estado

`COMM_STATE_CONNECTED`: Indica que el medio de comunicación ha establecido una conexión física o lógica. En una UART simple este estado puede estar siempre presente si el periférico está inicializado; en USBTMC o TCP/IP refleja que el *host* remoto ha establecido la sesión. La *Communication Task* consulta este estado antes de intentar transmitir o recibir, evitando operaciones sobre una interfaz no disponible.

- COMM_STATE_RX_ACTIVE:** Señala que el *hardware* de recepción está habilitado. Se activa al invocar `start_rx()` y se desactiva con `stop_rx()`. Cuando está activo, la ISR o el DMA pueden escribir en el buffer de recepción y generar eventos `IFACE_EVENT_RX_DATA_AVAILABLE`.
- COMM_STATE_TX_ACTIVE:** Indica que hay una transmisión en curso. Es activado por el driver al iniciar la transmisión física de un bloque (después de que la *Communication Task* haya cargado los datos en el buffer de transmisión mediante `cb_put()` y llamado a `start_tx()`). Se desactiva cuando la transmisión finaliza, momento en el cual se genera el evento `IFACE_EVENT_TX_COMPLETE`.
- COMM_STATE_ERROR:** Representa una condición de falla persistente que impide la operación normal de la interfaz. Se activa cuando ocurre un evento de error crítico (por ejemplo, `IFACE_EVENT_TX_ERROR_BUS_FAULT`) y permanece activo hasta que se ejecuta una operación de reinicio (`reset()`) o reinicialización (`deinit()` seguido de `init()`). Mientras este estado está activo, la *Communication Task* puede inhibir las operaciones de recepción y transmisión.

Modificación y visibilidad Los estados son modificados exclusivamente por el driver, ya sea como resultado de operaciones invocadas a través de los *callbacks* de la interfaz (`start_rx`, `stop_rx`, `start_tx`, `reset`) o como respuesta a condiciones de *hardware* detectadas en la ISR. Las capas superiores acceden al campo `state` en modo exclusivamente lectura. Esta restricción preserva al driver como única fuente de verdad sobre el estado del periférico y evita inconsistencias derivadas de modificaciones externas no coordinadas.

Cuando la *Communication Task* necesita realizar una operación, invoca el *callback* correspondiente y evalúa el resultado. Por ejemplo, una solicitud de transmisión puede retornar un código que indica éxito, que la interfaz está ocupada o que no hay datos pendientes, permitiendo a la FSM reintentar en el siguiente ciclo. De este modo, el estado interno del driver gobierna la viabilidad de cada operación sin exponer su lógica a las capas superiores.

22.2.3 Eventos de Comunicación y Error

Los eventos representan sucesos asincrónicos e instantáneos que el driver notifica a la *Communication Task*. A diferencia de los estados, que reflejan condiciones persistentes, los eventos son puntuales: una vez consumidos por la tarea, desaparecen del registro hasta que la condición que los genera vuelva a producirse.

Todos los eventos se codifican en el tipo `comm_iface_event_t`, una máscara de bits donde cada bit representa un tipo de suceso. Esta representación permite que múltiples eventos coexistan en el registro —por ejemplo, la llegada de datos y la detección de un error de paridad pueden señalizarse simultáneamente— y que la *Communication Task* los procese de manera eficiente mediante operaciones de enmascaramiento binario.

Eventos de Flujo Normal Son aquellos que informan del progreso ordinario de la comunicación:

IFACE_EVENT_INTERFACE_CONNECTED: La interfaz ha alcanzado un estado operativo y está lista para transmitir y recibir. En medios donde la conexión es inmediata (UART), el driver publica este evento al completar exitosamente `init()`. En medios negociados (USBTMC, TCP/IP), se publica cuando la negociación con el host remoto finaliza.

IFACE_EVENT_INTERFACE_DISCONNECTED: La conexión se ha perdido. Aplica a interfaces como USBTMC o TCP/IP, donde la sesión puede cerrarse.

IFACE_EVENT_RX_DATA_AVAILABLE: Se han recibido nuevos bytes y están disponibles en el buffer de recepción. Es la señal principal que dispara la FSM de recepción.

IFACE_EVENT_TX_COMPLETE: La transmisión en curso ha finalizado exitosamente. El hardware (DMA o periférico) ha enviado todos los bytes y el canal de transmisión está nuevamente disponible.

Eventos de Error Notifican condiciones anómalas detectadas durante la operación. Se dividen según la dirección del flujo afectado:

Errores de recepción:

- **IFACE_EVENT_RX_ERROR_OVERFLOW:** el buffer de recepción no pudo alojar todos los datos entrantes y se produjo pérdida de bytes.
- **IFACE_EVENT_RX_ERROR_FRAMING:** error de encuadre en la trama (UART).
- **IFACE_EVENT_RX_ERROR_PARITY:** error de paridad (UART).

Errores de transmisión:

- **IFACE_EVENT_TX_ERROR_TIMEOUT:** el *host* remoto no consumió los datos dentro del tiempo estipulado.
- **IFACE_EVENT_TX_ERROR_BUS_FAULT:** error físico en el bus de datos (por ejemplo, un fallo de DMA).

Error interno:

- **IFACE_EVENT_INTERNAL_ERROR:** condición anómala no vinculada directamente al medio, como un fallo de asignación de memoria o una inconsistencia interna del driver.

Relación con el estado de error La activación de un evento de error puede provocar adicionalmente la activación del estado persistente `COMM_STATE_ERROR` por parte del driver. Esta decisión queda a criterio de cada implementación concreta; en general, los errores que comprometen la operación futura de la interfaz (como un fallo de bus o un overflow que requiera reinicio del periférico) activan el estado de error, mientras que los errores recuperables (como un encuadre puntual) pueden limitarse a notificar el evento sin modificar el estado global.

Producción y consumo de eventos El driver escribe en el registro de eventos mediante el *callback* `set_event()`, típicamente desde la rutina de servicio de interrupción (ISR) o desde el código que atiende una condición de error detectada durante una operación solicitada (`start_tx`, `start_rx`). La *Communication Task* consume los eventos al inicio

de cada ciclo mediante el *callback* `get_event()`, que implementa una lectura atómica con limpieza inmediata (*clear-on-read*). Este mecanismo, detallado en la sección correspondiente del middleware, garantiza que ningún evento se pierda entre ciclos y que todas las FSM trabajen sobre una instantánea coherente del estado del *hardware*.

22.2.4 APIs de Control de Flujo

Los *callbacks* de control de flujo permiten a la *Communication Task* gestionar la recepción y la transmisión de datos a través de la interfaz física. Todas estas funciones son **no bloqueantes** y retornan un código de tipo `com_response_t` que informa del resultado inmediato de la operación solicitada.

Los códigos de retorno están definidos en el tipo `com_response_t`:

Cuadro 22.3: Valores definidos en `com_response_t`.

Símbolo	Valor	Interpretación para la FSM
COMM_IFACE_OK	0x00	Operación completada con éxito.
COMM_IFACE_ERROR	0x01	La interfaz está en condición de error y rechazó la operación.
COMM_IFACE_BUSY	0x02	La interfaz está ocupada y no puede aceptar la operación en este ciclo.
COMM_IFACE_IDLE	0x03	Sin datos pendientes (aplica a <code>start_tx</code>).

`start_rx()`

Firma: `com_response_t (*start_rx)(void *ctx);`

Propósito: Activar el canal de recepción del periférico y prepararlo para recibir datos desde el medio físico.

Comportamiento esperado: Configura el *hardware* para que los bytes entrantes se depositen directamente en el buffer de recepción inyectado (`rx_buffer`) mediante DMA o interrupciones. Si la configuración es exitosa, el driver activa el estado `COMM_STATE_RX_ACTIVE` y, opcionalmente, limpia un error previo. En caso de fallo, desactiva el estado de recepción, activa `COMM_STATE_ERROR` y publica el evento `IFACE_EVENT_INTERNAL_ERROR`.

Valores de retorno:

- `COMM_IFACE_OK`: la recepción se ha iniciado correctamente.
- `COMM_IFACE_ERROR`: no se pudo iniciar la recepción (fallo del periférico o inconsistencia interna).

`stop_rx()`

Firma: `com_response_t (*stop_rx)(void *ctx);`

Propósito: Detener la recepción en curso y liberar los recursos de *hardware* asociados.

Comportamiento esperado: Detiene el DMA o las interrupciones de recepción y desactiva el estado `COMM_STATE_RX_ACTIVE`. Si la interfaz no está disponible, publica `IFACE_EVENT_INTERNAL_ERROR`.

Valores de retorno:

- `COMM_IFACE_OK`: la recepción se detuvo correctamente.
- `COMM_IFACE_ERROR`: la interfaz no se encontró o hubo un error interno.

`start_tx()`

Firma: `com_response_t (*start_tx)(void *ctx);`

Propósito: Iniciar la transmisión de los datos acumulados en el buffer de salida (`tx_buffer`) hacia el medio físico.

Comportamiento esperado: El driver verifica el estado del *hardware* de transmisión y la disponibilidad de datos en `tx_buffer`:

- Si el *hardware* está ocupado (por ejemplo, una transmisión previa aún en curso), retorna `COMM_IFACE_BUSY`.
- Si el buffer `tx_buffer` está vacío, retorna `COMM_IFACE_IDLE`.
- Si el *hardware* está en condición de error, activa `COMM_STATE_ERROR`, publica un evento de error (por ejemplo, `IFACE_EVENT_TX_ERROR_BUS_FAULT`) y retorna `COMM_IFACE_ERROR`.
- Si el *hardware* está listo y hay datos, lee el bloque contiguo de bytes desde `tx_buffer` (respetando las protecciones de acceso concurrente), programa el DMA o la interrupción de transmisión, activa `COMM_STATE_TX_ACTIVE` y retorna `COMM_IFACE_OK`.

Valores de retorno:

- `COMM_IFACE_OK`: la transmisión se ha iniciado correctamente.
- `COMM_IFACE_BUSY`: el *hardware* está ocupado; la FSM reintentará en el siguiente ciclo.
- `COMM_IFACE_IDLE`: no hay datos pendientes en el buffer de salida.
- `COMM_IFACE_ERROR`: fallo irrecuperable en la transmisión; la FSM debe notificar el error y ejecutar `reset()` sobre el canal.

En todos los casos, el resultado retornado es el único criterio que utiliza la *Communication Task* para decidir la acción siguiente. Los eventos de error que el driver publique de manera asincrónica serán capturados en el siguiente *snapshot* y procesados por la FSM correspondiente con independencia del código de retorno.

22.2.5 Protección de Acceso Concurrente

Los buffers de recepción y transmisión son recursos compartidos entre dos contextos de ejecución: la *Communication Task* (que los lee y escribe en el ciclo principal) y la rutina de servicio de interrupción o el controlador DMA (que los escribe y lee de manera asincrónica). Para garantizar la integridad de los datos sin recurrir a mecanismos de bloqueo pesados, la interfaz `comm_iface_t` proporciona cuatro *callbacks* de protección que encapsulan la estrategia de sincronización específica de cada periférico.

Pares de protección

protect_rx() / unprotected_rx(): Protegen el acceso al buffer de recepción. La *Communication Task* invoca `protect_rx()` antes de leer datos con `cb_get()` y `unprotect_rx()` al finalizar. La ISR del driver invoca estas mismas funciones antes y después de escribir nuevos bytes en el buffer.

protect_tx() / unprotected_tx(): Protegen el acceso al buffer de transmisión. La *Communication Task* las utiliza cuando escribe datos con `cb_put()` antes de solicitar el inicio de la transmisión. El driver las emplea al leer datos desde `tx_buffer` para alimentar el DMA o el registro de transmisión.

Firma y semántica Las cuatro funciones comparten la misma firma:

```
void (*protect_...)(void *ctx);
```

El parámetro `ctx` permite que cada implementación acceda a su contexto privado (por ejemplo, para deshabilitar una interrupción específica o liberar un *spinlock*). Las capas superiores invocan estos *callbacks* sin conocer la estrategia concreta utilizada por el periférico.

Estrategias de implementación La naturaleza de la protección depende del periférico y del entorno de ejecución:

- En un microcontrolador *bare-metal* con una UART, la protección de recepción puede consistir en deshabilitar temporalmente la interrupción del periférico (`NVIC_DisableIRQ`) para evitar que la ISR modifique `head` mientras la tarea lee `tail`. La protección de transmisión puede deshabilitar la interrupción de DMA correspondiente.
- En un sistema con RTOS, puede emplearse un *mutex* o un *spinlock* que el driver almacena en su contexto.
- En periféricos con DMA que opera sobre descriptores encadenados, la protección puede ser innecesaria si el *hardware* garantiza la atomicidad de las transferencias.
- En un microcontrolador *bare-metal* con una UART, la protección de recepción consiste en deshabilitar temporalmente la interrupción del periférico (`NVIC_DisableIRQ`) para evitar que la ISR modifique `head` mientras la tarea lee `tail`. La protección de transmisión deshabilita la interrupción de DMA correspondiente (`__HAL_DMA_DISABLE_IT`).

En todos los casos, la *Communication Task* y la capa de aplicación permanecen completamente desacopladas de la estrategia de sincronización subyacente. Solo invocan los *callbacks* en los puntos definidos por el contrato de la interfaz.

Uso coordinado con las FSM La FSM de Recepción protege el buffer antes de iterar con `cb_get()` y libera la protección al terminar. La FSM de Transmisión protege el buffer antes de copiar los datos con `cb_put()` y libera la protección antes de invocar `start_tx()`, ya que el driver volverá a proteger el buffer internamente durante la lectura para el DMA. Esta delimitación precisa de las secciones críticas minimiza el tiempo de inhibición de interrupciones y evita la inversión de prioridad.

22.2.6 Estrategias de Sincronización y Gestión de Buffers

Los buffers de recepción y transmisión son buffers circulares propiedad de la *Communication Task* e inyectados en la estructura `comm_iface_t` antes de la inicialización. La tarea los manipula directamente mediante las funciones `cb_get()` y `cb_put()` provistas por la biblioteca de buffer circular, protegiendo los accesos con los *callbacks* `protect_rx()` y `protect_tx()` que el propio driver proporciona.

La ISR o el DMA escribe cada byte en el buffer circular protegido mediante `protect_rx()`, y avanza el índice de escritura (`head`).

El driver, por su parte, solo accede a estos buffers para dos operaciones elementales: escribir bytes recibidos en el buffer de recepción, y leer bytes a transmitir desde el buffer de transmisión. La estrategia con que lo hace depende de las capacidades del periférico.

Acceso Directo al Buffer Circular (con Protección ISR)

En periféricos que generan una interrupción por cada byte recibido o que disponen de DMA con libertad de escritura sobre un rango de memoria, el driver opera directamente sobre el buffer circular inyectado:

- En **recepción**, la ISR o el DMA escribe cada byte en el buffer circular y avanza el índice de escritura (`head`). La *Communication Task* lee con `cb_get()` y avanza el índice de lectura (`tail`).
- En **transmisión**, la *Communication Task* deposita los datos con `cb_put()`. Cuando se invoca `start_tx()`, el driver lee desde el buffer circular, programa el DMA y avanza el índice de lectura (`tail`) a medida que el hardware consume los bytes.

La detección de desbordamiento en recepción ocurre en la ISR: si al llegar un nuevo byte el buffer circular está lleno, se genera el evento `IFACE_EVENT_RX_ERROR_OVERFLOW`. En transmisión, la *Communication Task* no escribe si su propio buffer está lleno (lo sabe mediante `cb_status()`), por lo que no se produce desbordamiento.

Doble Buffer Interno del Driver

Cuando el periférico no permite acceso DMA directo al buffer circular, o trabaja por bloques, el driver puede implementar un doble buffer como mecanismo auxiliar interno. En este esquema:

- El driver reserva dos buffers locales (no expuestos a la *Communication Task*). El hardware escribe en uno mientras el otro se transfiere al buffer circular inyectado.
- Al completar un bloque, el driver intercambia los buffers y copia los datos recibidos al buffer circular de recepción. Durante esta copia se aplican las protecciones necesarias.
- En transmisión, el driver copia datos desde el buffer circular inyectado a un buffer local de trabajo antes de iniciar la transmisión por hardware.

Para la *Communication Task*, la existencia del doble buffer es transparente: sigue viendo un único buffer circular sobre el cual aplica `cb_get()` y `cb_put()` como en el caso anterior. La protección de acceso se limita a las secciones críticas durante las copias entre el buffer circular y los buffers internos del driver.

Independencia de la Estrategia La *Communication Task* desconoce por completo si el driver accede directamente al buffer circular o utiliza un doble buffer interno. Su interacción se reduce a: proteger el buffer (`protect_rx/protect_tx`), leer o escribir con `cb_get/cb_put`, y liberar la protección. Esta uniformidad permite que el middleware opere sin cambios sobre distintos periféricos.

22.3 Capa de Middleware (Communication Task)

La capa de middleware constituye el núcleo del subsistema de comunicación. Se implementa como una tarea independiente —denominada *Communication Task*— que opera sobre la interfaz abstracta `comm_iface_t` proporcionada por el driver. Su responsabilidad es coordinar el ciclo de vida de la interfaz, delimitar mensajes a partir del flujo de bytes, gestionar la transmisión de respuestas y ofrecer a la capa de aplicación una API basada en mensajes completos.

22.3.1 Modelo de Ejecución y Planificación

La *Communication Task* está diseñada para ejecutarse en un entorno cooperativo, ya sea como un hilo independiente dentro de un RTOS o como un bucle dentro de un esquema *super-loop* en sistemas *bare-metal*. En ambos casos, el modelo de ejecución respeta los siguientes principios:

- **Ejecución no bloqueante:** Todas las FSM evalúan sus entradas —el *snapshot* de eventos y el resultado de las operaciones invocadas— y retornan el control sin esperas activas. Si una operación no puede completarse (por ejemplo, `start_tx()` retorna `COMM_IFACE_BUSY`), la FSM simplemente difiere la acción al siguiente ciclo.
- **Temporización simple:** Los tiempos de espera máximos (*timeouts*) para el hardware o la recepción de mensajes incompletos se gestionan mediante una única base de tiempo del sistema (`sys_tick`) con resolución de milisegundos.
- **Determinismo:** Al ejecutar todas las fases de comunicación en un único contexto de manera secuencial, se eliminan por diseño las condiciones de carrera entre las FSM. La única sección crítica que requiere protección externa es la captura del *snapshot* de eventos, que se resuelve de manera atómica en el driver.

El costo computacional de evaluar todas las FSM en cada ciclo es insignificante, ya que cada una aplica una máscara binaria sobre el *snapshot* y, si no hay eventos de su competencia, retorna de inmediato.

22.3.2 Registro de Eventos y Captura Atómica (Snapshot)

El driver mantiene un registro de eventos en forma de máscara de bits (`comm_iface_event_t`) que es escrito de manera asíncrona por la ISR o por las propias funciones del driver. Para que la *Communication Task* pueda operar sobre una base de eventos coherente y sin condiciones de carrera, se implementa el siguiente procedimiento al inicio de cada ciclo:

1. **Captura atómica:** La tarea invoca el *callback* `get_event()` de la interfaz activa. Este *callback*, ejecutado dentro de una sección crítica, lee el valor actual del registro de eventos y lo pone a cero (*clear-on-read*) en la misma operación. El valor leído se almacena en una variable local denominada *snapshot de eventos*.
2. **Procesamiento sobre copia local:** Durante el resto del ciclo, ninguna FSM vuelve a acceder al registro físico del driver. Todas las máquinas de estado trabajan exclusivamente sobre el *snapshot*, aplicando máscaras binarias para aislar los eventos que les competen.
3. **Nuevos eventos:** Si durante el ciclo se produce un nuevo evento de hardware, la ISR lo escribirá en el registro del driver (ahora limpio), y será capturado en el *snapshot* del ciclo siguiente.

Este mecanismo garantiza la coherencia temporal: todas las FSM evalúan exactamente el mismo estado del hardware para un mismo ciclo de ejecución, sin que los eventos procesados por una máquina interfieran en las decisiones de las demás.

22.3.3 Orden de Precedencia y Flujo de Control

El *snapshot* de eventos es evaluado por las FSM en un orden fijo que refleja la criticidad de cada función. El bucle de la *Communication Task* sigue la secuencia:

1. **FSM de Gestión de Interfaz (prioridad máxima):** Procesa los eventos de cambio de conexión y los errores críticos del hardware. Si se detecta un fallo que compromete la integridad de la interfaz, esta FSM toma el control, ejecuta las rutinas de mitigación o reinicio y puede alterar el estado global antes de que las demás máquinas evalúen sus eventos.
2. **FSM de Recepción (RX):** Se ejecuta a continuación. Si la interfaz está operativa, procesa los eventos `IFACE_EVENT_RX_DATA_AVAILABLE` y los errores de recepción, extrayendo bytes del buffer circular y delimitando mensajes.
3. **FSM de Transmisión (TX):** Procesa los eventos relacionados con la finalización de envíos (`IFACE_EVENT_TX_COMPLETE`) y los errores de transmisión. También atiende las solicitudes de transmisión generadas internamente por la capa de aplicación.

El flujo macroscópico del bucle operativo es, por tanto:

Snapshot → FSM Gestión → FSM RX → FSM TX

Si la capa de aplicación se ejecuta en el mismo contexto, la FSM de Procesamiento SCPI se invoca a continuación, fuera ya del ámbito del middleware de comunicación.

22.3.4 Máquinas de Estado de Transporte

Las tres máquinas de estado que gobiernan el comportamiento del middleware se modelan según el formalismo de Mealy: las transiciones y las acciones dependen del estado actual y de los eventos de entrada. Los estados son estrictamente privados y la comunicación entre máquinas se realiza mediante la publicación de eventos lógicos internos.

FSM de Gestión de Interfaz

Esta máquina supervisa el estado global de la interfaz, coordina las transiciones operativas, la recuperación ante fallos y, cuando es necesario, la selección de un nuevo medio de transporte.

Cuadro 22.4: Tabla de transiciones de la FSM de Gestión de Interfaz.

Estado Actual	Evento	Siguiente Estado	Acción
DOWN	CMD_START	READY	init()
READY	IFACE_EVENT_INTERFACE_CONNECTED	ACTIVE	start_rx()
ACTIVE	CMD_DISABLE_RX	READY	stop_rx()
ACTIVE	Evento de error crítico	ERROR	Publicar condición de falla
ERROR	CMD_RESET	READY	reset()
ERROR	Error irreparable / cambio requerido	SELECTING	deinit() actual; invocar selector
SELECTING	Interfaz disponible	READY	init() sobre nueva interfaz
SELECTING	Sin interfaces disponibles	DOWN	Notificar indisponibilidad

Cuando la FSM transita a SELECTING, invoca al *selector de medio de transporte* —un componente independiente cuyo contrato se describe en la sección 22.3.7— para que determine cuál es la siguiente interfaz disponible. Si el selector provee una nueva instancia de `comm_iface_t`, la FSM la adopta como interfaz activa y ejecuta su `init()`. Si no hay interfaces alternativas, la FSM retorna a DOWN y notifica la indisponibilidad del subsistema.

FSM de Recepción (RX)

La FSM de Recepción consume los eventos de recepción del *snapshot* y extrae los bytes del buffer circular `rx_buffer` mediante `cb_get()`. Su responsabilidad es delimitar mensajes completos detectando el terminador LF.

Los caracteres CR (0x0D) se descartan silenciosamente durante el proceso de ensamblado, en conformidad con IEEE 488.2. La FSM no interactúa directamente con el hardware ni ejecuta rutinas de recuperación física; esa responsabilidad recae en la FSM de Gestión.

Cuadro 22.5: Tabla de transiciones de la FSM de Recepción.

Estado Actual	Evento	Siguiente Estado	Acción
RX_IDLE	IFACE_EVENT_RX_DATA_AVAILABLE	RX_GATHERING	Proteger buffer y leer bytes
RX_GATHERING	Byte distinto de LF	RX_GATHERING	Acumular carácter en buffer de mensaje
RX_GATHERING	Byte LF	RX_IDLE	Publicar EV_MESSAGE_READY
RX_GATHERING	Error de RX	RX_IDLE	Publicar error y liberar buffer

FSM de Transmisión (TX)

Esta máquina administra el flujo de salida de datos. Responde a las solicitudes de transmisión de la capa de aplicación y gestiona el envío de los datos acumulados en `tx_buffer` hacia el driver.

Cuadro 22.6: Tabla de transiciones de la FSM de Transmisión.

Estado Actual	Evento	Siguiente Estado	Acción
TX_IDLE	EV_TX_REQUEST	TX_BUSY	Invocar <code>start_tx()</code>
TX_BUSY	COMM_IFACE_OK	TX_BUSY	Esperar TX_COMPLETE
TX_BUSY	COMM_IFACE_BUSY	TX_BUSY	Reintentar en siguiente ciclo
TX_BUSY	COMM_IFACE_IDLE	TX_IDLE	Sin datos pendientes
TX_BUSY	COMM_IFACE_ERROR	TX_IDLE	Publicar error de TX
TX_BUSY	IFACE_EVENT_TX_COMPLETE	TX_IDLE	Publicar EV_TX_DONE
TX_BUSY	IFACE_EVENT_TX_ERROR...	TX_IDLE	Ejecutar <code>reset()</code> y publicar error

La máquina evalúa el código de retorno de `start_tx()` para decidir la acción inmediata, sin consultar el estado de la interfaz. Los eventos de finalización y error se reciben a través del *snapshot* en ciclos posteriores.

22.3.5 API de Servicios hacia la Capa de Aplicación

La *Communication Task* expone a la capa de aplicación (Motor SCPI) una interfaz basada en mensajes completos, ocultando por completo los detalles del transporte. Esta API consta de dos eventos lógicos y una función de solicitud de transmisión:

EV_MESSAGE_READY: Evento publicado por la FSM de Recepción cuando se ha delimitado un mensaje completo (terminado en LF) en el buffer de recepción. La aplicación debe consumir el mensaje antes del siguiente ciclo de la tarea, copiándolo a sus buffers propios.

send_message(data, len): Función invocada por la aplicación para solicitar el envío de una respuesta. La *Communication Task* copia los datos al buffer de transmisión mediante `cb_put()` y activa internamente la señal `EV_TX_REQUEST`, que será evaluada por la FSM de Transmisión en el ciclo siguiente.

EV_TX_DONE: Evento publicado por la FSM de Transmisión cuando la transmisión se ha completado exitosamente. Permite a la aplicación liberar recursos asociados a la respuesta o encolar un nuevo mensaje.

La aplicación no accede directamente al hardware de comunicación ni a los buffers de transporte. Toda la interacción se realiza a través de estos tres puntos de contacto.

22.3.6 Adaptación a Protocolos de Instrumentación

La *Communication Task* es responsable de la delimitación de mensajes de acuerdo con las reglas del protocolo de instrumentación aplicable, pero no interpreta su contenido.

Delimitación de mensajes La FSM de Recepción identifica el fin de mensaje al detectar el carácter LF (0x0A), en conformidad con la norma IEEE 488.2. Los caracteres CR (0x0D) que preceden opcionalmente al terminador se descartan de manera silenciosa. El resultado es un mensaje completo —una secuencia de bytes delimitada— que se entrega a la capa de aplicación sin modificación alguna.

Independencia del protocolo El middleware no distingue entre un comando SCPI, una consulta de estado o cualquier otra carga útil. Opera exclusivamente con mensajes como secuencias de bytes delimitadas. Esta independencia permite que el mismo código de middleware dé servicio a distintos protocolos de aplicación (SCPI, Modbus, propietario) sin más cambio que el motor de procesamiento conectado a su API.

Interrupciones de consulta (*Query Interrupts*) Si el protocolo del periférico subyacente dispone de mecanismos de interrupción de consulta —como el control de flujo en USBTMC—, el driver traduce dicha condición de hardware en un evento abstracto de interfaz. La FSM de Gestión captura este evento y coordina la acción correspondiente evaluando el estado operativo actual, sin que el significado del comando trascienda a esta capa.

22.3.7 Selector de Medio de Transporte

La *Communication Task* opera exclusivamente sobre la interfaz abstracta `comm_iface_t`. La asociación con una implementación concreta es resuelta por el **selector de medio de transporte**, un componente independiente cuya responsabilidad es determinar, en cada momento, cuál de las interfaces físicas disponibles debe estar activa.

El selector es invocado en dos circunstancias:

- Durante la inicialización del subsistema, para establecer la primera interfaz activa.
- Desde la FSM de Gestión de Interfaz, cuando se produce un error irrecuperable o se requiere un cambio explícito de medio.

El selector evalúa la disponibilidad de cada implementación registrada, aplica criterios de prioridad o configuración del instrumento, y retorna una referencia a la ins-

tancia de `comm_iface_t` seleccionada, o un valor nulo si no hay ninguna disponible. No participa en la transferencia ni en el procesamiento de datos.

23

Motor SCPI

[Motor SCPI]

Este módulo consume la API de mensajes del Sistema de Comunicación...

23.1 Integración del parser libscpi

23.2 Flujo de recepción y análisis de comandos

23.3 Generación y transmisión de respuestas

Anexos

